Khushi Shukla-D15A/62

Experiment No 7 : Customer Churn Prediction using Artificial Neural Network (ANN)

Aim

To design and implement an Artificial Neural Network (ANN) using Keras and TensorFlow to predict whether a customer will leave a bank (churn prediction) based on customer attributes.

Theory

An **Artificial Neural Network (ANN)** is a machine learning model inspired by the human brain. It consists of multiple layers:

- **Input Layer**: Takes input features
- **Hidden Layers**: Learn patterns and relationships
- **Output Layer**: Produces prediction

Each neuron computes:

$$z = \sum (w_i x_i) + b$$

$$a = f(z)$$

Where:

- w_i: weights
- x_i: inputs
- b: bias
- $f(z)$: activation function

Activation Functions:

- ReLU: $f(x) = \max(0, x)$
- Sigmoid (used for binary classification):

$$f(x) = \frac{1}{1 + e^{-x}}$$

Loss Function (Binary Crossentropy):

$$L = -\frac{1}{N} \sum [y \log(\hat{y}) + (1 - y) \log(1 - \hat{y})]$$

1. Dataset Source

- **Dataset Name**: Churn Modelling Dataset

- **Source**: Kaggle
- **Link**: https://www.kaggle.com/datasets/shrutimechlearn/churn-modelling

2. Dataset Description

- **Total Records**: 10,000
- **Features**: 10 input features after preprocessing
- **Target Variable**: Exited
    - 1 → Customer left (Churn)
    - 0 → Customer stayed

Features:

- CreditScore
- Geography
- Gender
- Age
- Tenure
- Balance
- NumOfProducts
- HasCrCard
- IsActiveMember
- EstimatedSalary

Characteristics:

- Binary classification problem
- Contains categorical + numerical data
- Slightly imbalanced dataset (more non-churn cases)

3. Mathematical Formulation of ANN

Forward Propagation:

$$z^{(l)} = W^{(l)} a^{(l-1)} + b^{(l)}$$

$$a^{(l)} = f(z^{(l)})$$

Loss Function:

$$L = -[y \log(\hat{y}) + (1 - y) \log(1 - \hat{y})]$$

Backpropagation:

$$W = W - \eta \frac{\partial L}{\partial W}$$

Where:

- η: learning rate

4. Algorithm Limitations

- Requires large dataset for better performance
- Sensitive to hyperparameters
- Black-box model (low interpretability)
- Can suffer from overfitting
- Performance affected by class imbalance

5. Methodology / Workflow

Steps:

1. Load dataset
2. Data preprocessing
    - Remove irrelevant columns
    - Encode categorical variables
    - Feature scaling
3. Train-test split
4. Build ANN model
5. Compile model
6. Train model
7. Evaluate performance

Workflow Diagram:

Dataset → Cleaning → Encoding → Scaling → ANN Model → Training → Evaluation

6. Code Implementation

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.metrics import confusion_matrix, classification_report

from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Dropout
from tensorflow.keras.callbacks import EarlyStopping
from tensorflow.keras.optimizers import Adam
from tensorflow.keras import Input

df = pd.read_csv("Churn_Modelling.csv")
df = df.drop(['RowNumber', 'CustomerId', 'Surname'], axis=1)
le_gender = LabelEncoder()
df['Gender'] = le_gender.fit_transform(df['Gender'])

# One-hot encoding for Geography
df = pd.get_dummies(df, columns=['Geography'], drop_first=True)

# Features & target
X = df.drop('Exited', axis=1)
y = df['Exited']

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
# Feature scaling
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# ========================
# BUILD ANN MODEL
# ========================
model = Sequential([
    Input(shape=(X_train.shape[1],)),
    Dense(64, activation='relu'),
    Dropout(0.3),
    Dense(32, activation='relu'),
    Dense(1, activation='sigmoid')   # Binary output
])
# ========================
# COMPILE MODEL
# ========================
model.compile(
    optimizer=Adam(learning_rate=0.001),
    loss='binary_crossentropy',
    metrics=['accuracy']
)
# ========================
# TRAIN MODEL
# ========================
early_stop = EarlyStopping(
```

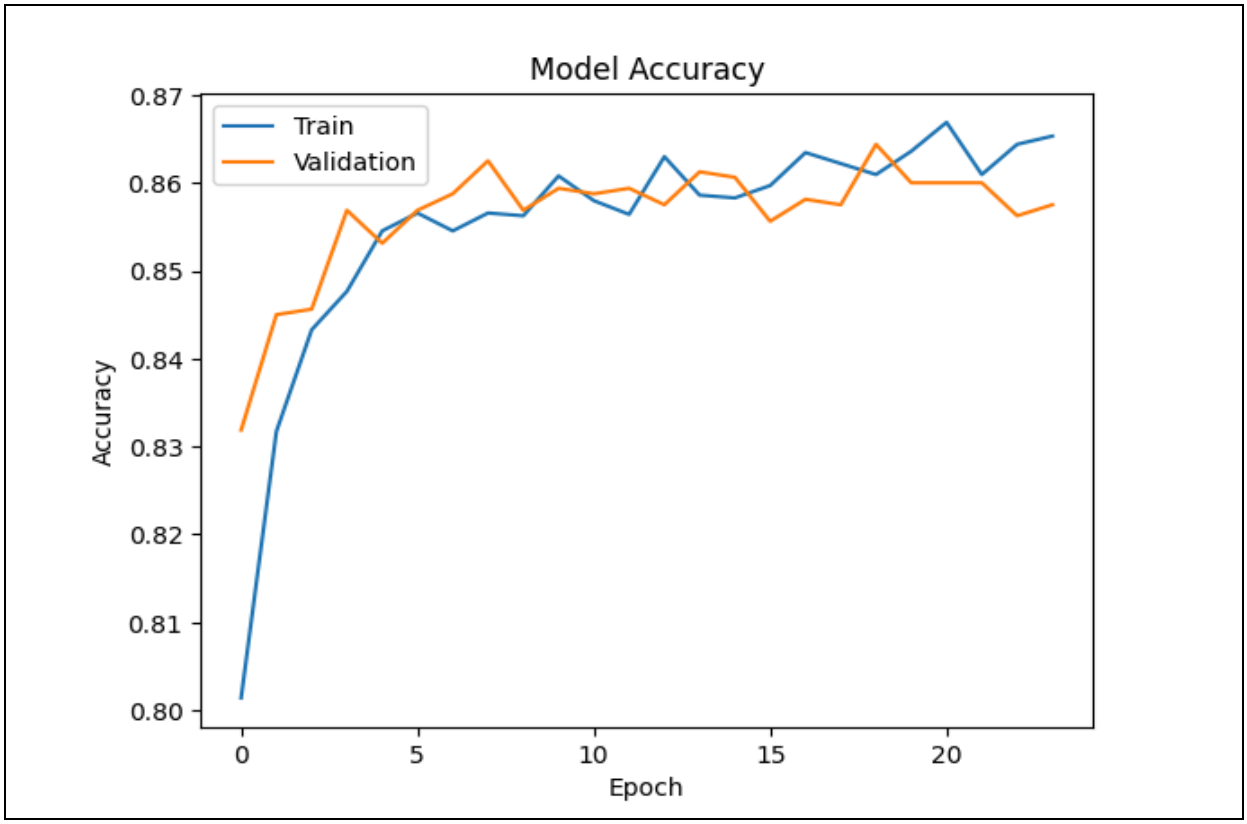
```
    monitor='val_loss',
    patience=5,
    restore_best_weights=True
)
history = model.fit(
    X_train, y_train,
    epochs=50,
    batch_size=16,
    validation_split=0.2,
    callbacks=[early_stop]
)
# ==========================
# EVALUATE MODEL
# ==========================
loss, accuracy = model.evaluate(X_test, y_test)
print("\nTest Accuracy:", accuracy)

# ==========================
# PREDICTIONS
# ==========================
y_pred = model.predict(X_test)
y_pred = (y_pred > 0.5).astype(int)

# ==========================
# CONFUSION MATRIX
# ==========================
cm = confusion_matrix(y_test, y_pred)

plt.figure(figsize=(6, 5))
sns.heatmap(cm, annot=True, fmt='d')
plt.title("Confusion Matrix")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()

# ==========================
# CLASSIFICATION REPORT
# ==========================
print("\nClassification Report:\n")
print(classification_report(y_test, y_pred))

# ==========================
# ACCURACY GRAPH
# ==========================
plt.figure()
plt.plot(history.history['accuracy'])
plt.plot(history.history['val_accuracy'])
plt.title('Model Accuracy')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.legend(['Train', 'Validation'])
plt.show()

# ==========================
```

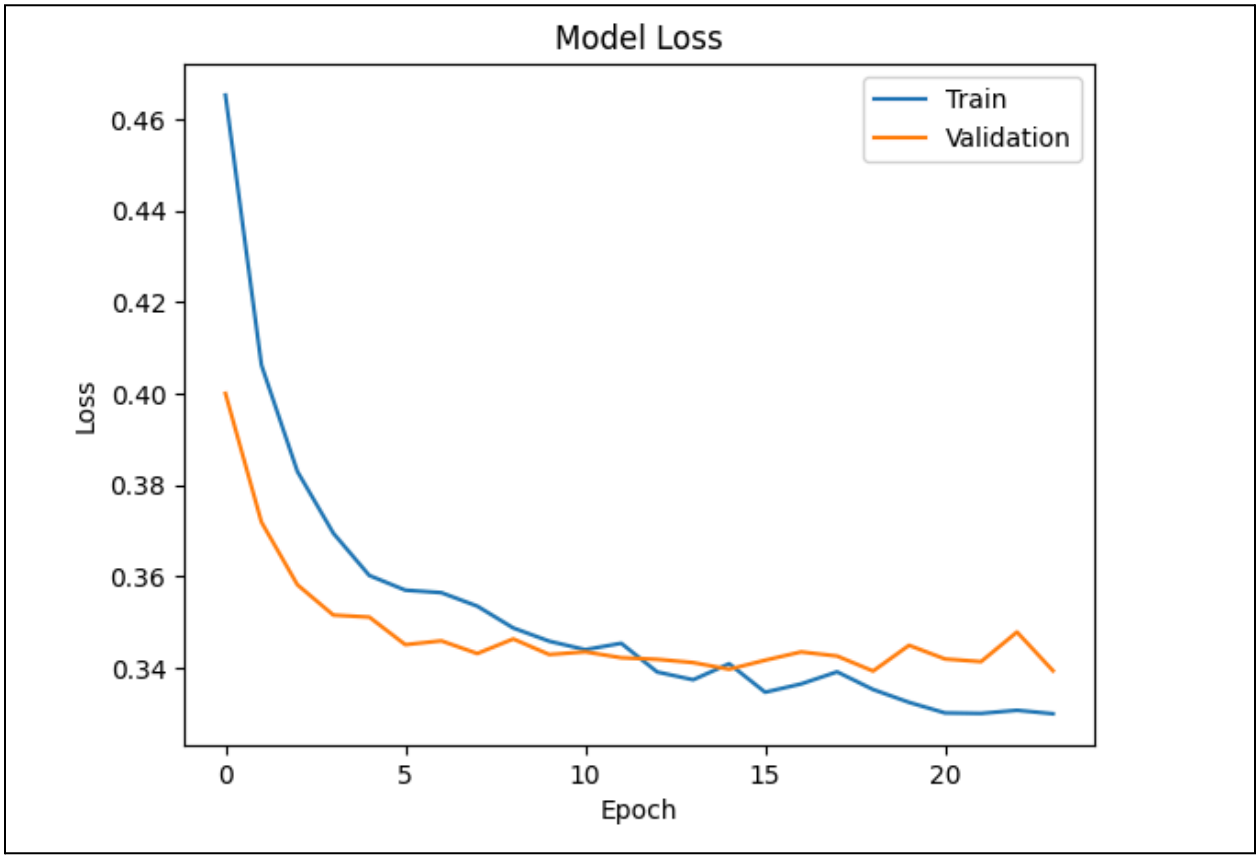
```
# LOSS GRAPH
# ========================
plt.figure()
plt.plot(history.history['loss'])
plt.plot(history.history['val_loss'])
plt.title('Model Loss')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.legend(['Train', 'Validation'])
plt.show()
```

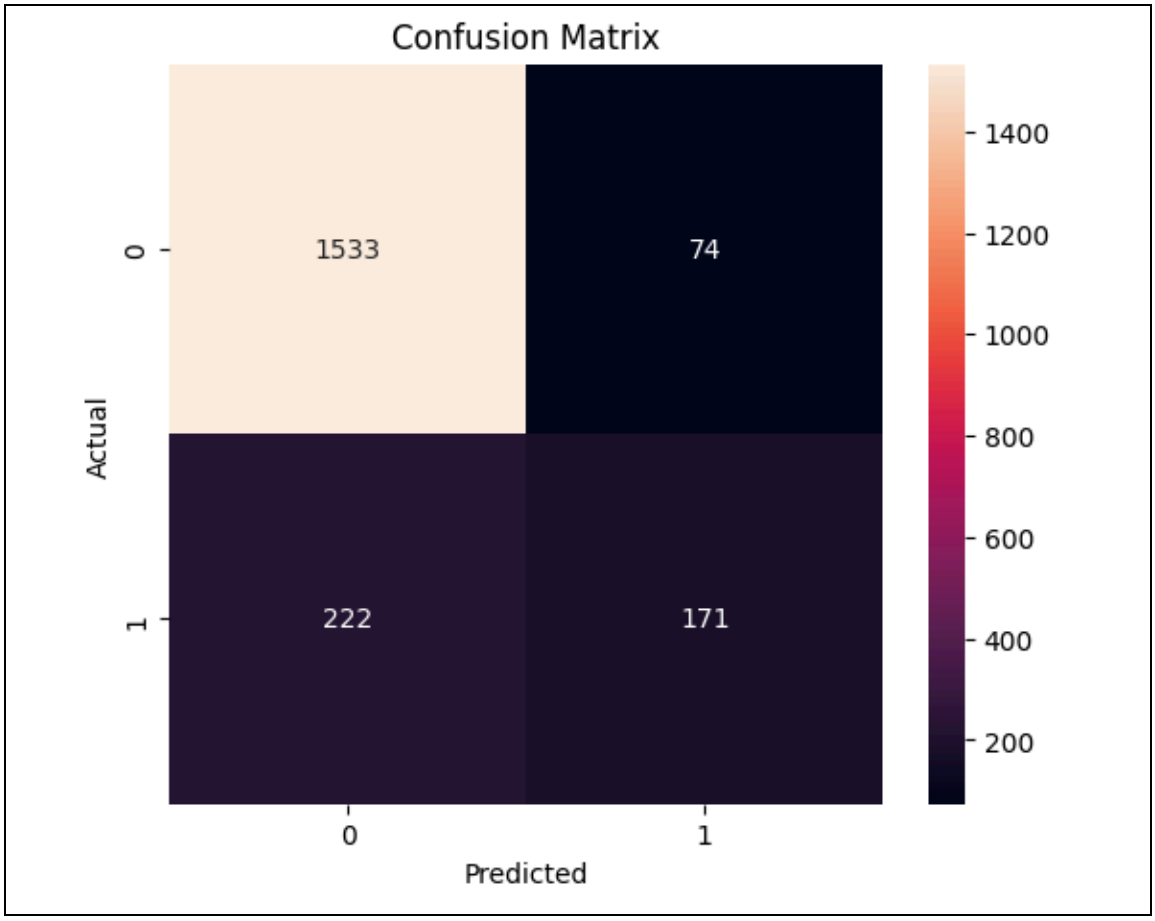
7. Output

```
Classification Report:

              precision    recall  f1-score   support

           0       0.87      0.95      0.91      1607
           1       0.70      0.44      0.54       393

    accuracy                           0.85      2000
   macro avg       0.79      0.69      0.72      2000
weighted avg       0.84      0.85      0.84      2000
```

```
400/400 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.8622 - loss: 0.3392 - val_accuracy: 0.8575 - val_loss: 0.3426
Epoch 19/50
400/400 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.8609 - loss: 0.3353 - val_accuracy: 0.8644 - val_loss: 0.3393
Epoch 20/50
400/400 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.8636 - loss: 0.3325 - val_accuracy: 0.8600 - val_loss: 0.3450
Epoch 21/50
400/400 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.8669 - loss: 0.3302 - val_accuracy: 0.8600 - val_loss: 0.3420
Epoch 22/50
400/400 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.8609 - loss: 0.3301 - val_accuracy: 0.8600 - val_loss: 0.3414
Epoch 23/50
400/400 ━━━━━━━━━━━━━━━━━━━━ 1s 1ms/step - accuracy: 0.8644 - loss: 0.3308 - val_accuracy: 0.8562 - val_loss: 0.3479
Epoch 24/50
400/400 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.8653 - loss: 0.3300 - val_accuracy: 0.8575 - val_loss: 0.3394
63/63 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - accuracy: 0.8520 - loss: 0.3387

✅ Test Accuracy: 0.8519999980926514
63/63 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step
```


Model Accuracy
Train
Validation
Accuracy
0.87
0.86
0.85
0.84
0.83
0.82
0.81
0.80
0
5
10
15
20
Epoch


Model Loss
Train
Validation
Loss
0.46
0.44
0.42
0.40
0.38
0.36
0.34
0
5
10
15
20
Epoch




```
Epoch 1/50
400/400 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.8014 - loss: 0.4652 - val_accuracy: 0.8319 - val_loss: 0.4000
Epoch 2/50
400/400 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.8317 - loss: 0.4062 - val_accuracy: 0.8450 - val_loss: 0.3719
Epoch 3/50
400/400 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.8433 - loss: 0.3830 - val_accuracy: 0.8456 - val_loss: 0.3582
Epoch 4/50
400/400 ━━━━━━━━━━━━━━━━━━━━ 1s 1ms/step - accuracy: 0.8477 - loss: 0.3694 - val_accuracy: 0.8569 - val_loss: 0.3516
Epoch 5/50
400/400 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.8545 - loss: 0.3602 - val_accuracy: 0.8531 - val_loss: 0.3511
Epoch 6/50
400/400 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.8566 - loss: 0.3570 - val_accuracy: 0.8569 - val_loss: 0.3451
Epoch 7/50
400/400 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.8545 - loss: 0.3565 - val_accuracy: 0.8587 - val_loss: 0.3459
Epoch 8/50
400/400 ━━━━━━━━━━━━━━━━━━━━ 1s 1ms/step - accuracy: 0.8566 - loss: 0.3535 - val_accuracy: 0.8625 - val_loss: 0.3432
Epoch 9/50
400/400 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.8562 - loss: 0.3488 - val_accuracy: 0.8569 - val_loss: 0.3464
Epoch 10/50
400/400 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.8608 - loss: 0.3459 - val_accuracy: 0.8594 - val_loss: 0.3430
Epoch 11/50
400/400 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.8580 - loss: 0.3440 - val_accuracy: 0.8587 - val_loss: 0.3435
Epoch 12/50
400/400 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.8564 - loss: 0.3454 - val_accuracy: 0.8594 - val_loss: 0.3422
Epoch 13/50
400/400 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.8630 - loss: 0.3391 - val_accuracy: 0.8575 - val_loss: 0.3419
Epoch 14/50
400/400 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.8586 - loss: 0.3374 - val_accuracy: 0.8612 - val_loss: 0.3412
Epoch 15/50
400/400 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.8583 - loss: 0.3409 - val_accuracy: 0.8606 - val_loss: 0.3397
Epoch 16/50
400/400 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.8597 - loss: 0.3347 - val_accuracy: 0.8556 - val_loss: 0.3417
Epoch 17/50
400/400 ━━━━━━━━━━━━━━━━━━━━ 1s 1ms/step - accuracy: 0.8634 - loss: 0.3365 - val_accuracy: 0.8581 - val_loss: 0.3435
Epoch 18/50
400/400 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.8622 - loss: 0.3392 - val_accuracy: 0.8575 - val_loss: 0.3426
Epoch 19/50
400/400 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.8609 - loss: 0.3353 - val_accuracy: 0.8644 - val_loss: 0.3393
Epoch 20/50
```

8. Performance Analysis

- The model achieved **85% accuracy**, which is considered good for this dataset
- Training and validation accuracy are close → minimal overfitting
- Loss decreases steadily → good convergence

Classification Report Insights:

- **Non-churn customers (0)**:
  - High precision and recall (~90%+)
- **Churn customers (1)**:
  - Lower recall (~44%)

Interpretation:

- Model performs better on majority class
- Struggles with minority class due to imbalance

9. Hyperparameter Tuning

Parameters Tuned:

- Number of hidden layers
- Neurons per layer
- Learning rate
- Batch size
- Epochs
- Dropout rate

Example Comparison:

Parameter	**Initial**	**Tuned**
Epochs	20	50
Batch Size	32	16

Hidden Layers	1	2
Dropout	0	0.3

Impact:

- Improved generalization
- Reduced overfitting
- Better validation accuracy

Conclusion

In this experiment, an Artificial Neural Network was successfully implemented to predict customer churn. The model achieved an accuracy of approximately 85%, demonstrating its effectiveness in handling real-world classification problems. However, due to class imbalance, the model showed lower performance in predicting churned customers. This can be improved using techniques like class weighting or resampling. Overall, ANN proved to be a powerful tool for customer behavior prediction and can be applied in banking systems for retention strategies.